

**HỘI THẢO KHOA HỌC
CÁC TRƯỜNG THPT CHUYÊN
TRẠI HÈ PHƯƠNG NAM LẦN THỨ VIII - NĂM 2025**



Chuyên đề
**GIẢI CÁC BÀI TOÁN TỔ HỢP HIỆU QUẢ VỚI
BACKTRACKING VÀ BITMASK**

- Lĩnh vực: Tin học



Năm thực hiện 2025

MỤC LỤC

MỤC LỤC	1
DANH MỤC CÁC BẢNG	1
DANH MỤC CÁC HÌNH	2
1. Giới thiệu	1
2. Một số khái niệm cơ bản	2
2.1 Kiến thức liên quan	2
2.2 Lý thuyết về xử lý bit.....	2
2.2.1. Các phép toán trên bit	2
2.2.2. Một số thao tác xử lý bit [1]	3
2.3 Phương pháp Quay Lui (Backtracking) giải các bài toán tổ hợp....	6
2.3.1 Thuật toán Quay Lui [2]	6
2.4 Phương pháp sinh dùng bitmask.....	7
2.4.1 Khái niệm về Bitmask.....	7
2.4.2 Thuật toán sinh tổ hợp bằng Bitmask	8
3. Một số bài tập áp dụng	8
3.1 Bài tập 1: Liệt kê dãy nhị phân (0,1) độ dài N.	8
3.2 Bài tập 2: Liệt kê tổ hợp	12
3.3 Bài tập 3: Liệt kê chỉnh hợp.....	16
3.4 Bài tập 4: Bài toán máy rút tiền tự động ATM.	21
3.5 Bài tập 5: Xếp N quân hậu.....	26
4. Một số bài toán khác trong sách giáo khoa chuyên tin quyển 1	35
5. Kết luận	35
TÀI LIỆU THAM KHẢO	36

DANH MỤC CÁC BẢNG

Bảng 2.1 – Mô tả các phép toán xử lý trên bit.....	2
Bảng 2.2 – Số thập phân và nhị phân để biểu diễn tập con tương ứng.	7
Bảng 3.1 - Mô phỏng chạy thuật toán liệt kê dãy nhị phân bằng bitmask.	11
Bảng 3.2 – So sánh hai thuật toán quay lui và bitmask trong bài liệt kê dãy nhị phân.	12
Bảng 3.3 - Mô phỏng chạy thuật toán liệt kê tập con k phần tử trong tập N phần tử bằng bitmask.	16
Bảng 3.4 – So sánh hai thuật toán quay lui và bitmask trong bài liệt kê tổ hợp.....	16
Bảng 3.5 – Mô phỏng chạy thuật toán liệt kê chỉnh hợp bằng bitmask.	20
Bảng 3.6 - So sánh hai thuật toán quay lui và bitmask trong bài toán liệt kê chỉnh hợp.....	21
Bảng 3.7 - Mô phỏng chạy thuật toán máy rút tiền tự động ATM bằng bitmask.	25
Bảng 3.8 - So sánh hai thuật toán quay lui và bitmask trong bài toán máy rút tiền tự động ATM.	26
Bảng 3.9 - So sánh hai thuật toán quay lui và bitmask trong bài toán quan hậu.....	35

DANH MỤC CÁC HÌNH

Hình 3.1 - Cây tìm kiếm quay lui trong bài toán liệt kê dãy nhị phân độ dài N [2].	10
Hình 3.2 - Cây tìm kiếm quay lui trong bài liệt kê tập con k phần tử của tập n phần tử.	14
Hình 3.3 - Cây tìm kiếm quay lui trong bài Liệt kê các chỉnh hợp không lặp chập k phần tử của tập N phần tử.	19
Hình 3.4 - Cây tìm kiếm quay lui trong bài máy rút tiền tự động ATM.	23
Hình 3.5 - Thể hiện một phương án đặt 8 quân hậu trên bàn cờ kích thước 8×8	27
Hình 3.6 - Thể hiện đường chéo chính và đường chéo phụ của bàn cờ kích thước 8×8	27
Hình 3.7 - Cây tìm kiếm quay lui trong bài xếp 8 hậu trong bàn cờ kích thước 8×8	30

1. Giới thiệu

Trong chương trình chuyên Tin học lớp 10, việc hình thành và phát triển tư duy thuật toán, cùng với khả năng phân tích và thiết kế lời giải cho các bài toán tổ hợp, là một mục tiêu trọng yếu. Đặc biệt, đối với những bài toán yêu cầu tìm kiếm lời giải tối ưu trong không gian trạng thái rất lớn – nơi số lượng phương án có thể lên tới hàng trăm nghìn, thậm chí hàng triệu – thì việc lựa chọn được chiến lược giải quyết phù hợp đóng vai trò quyết định đến độ chính xác, hiệu quả và tối ưu hóa của thuật toán.

Trong số các phương pháp giải quyết bài toán tổ hợp, **đệ quy quay lui (Backtracking)** và **kỹ thuật Bitmask** là hai công cụ nền tảng, có tính ứng dụng cao và mang lại hiệu quả rõ rệt:

- **Đệ quy quay lui** là một kỹ thuật cổ điển nhưng mạnh mẽ, cho phép xây dựng lời giải theo từng bước bằng cách thử và loại bỏ sớm các nhánh không khả thi. Phương pháp này đặc biệt hiệu quả trong các bài toán có cấu trúc ràng buộc rõ ràng, giúp giảm thiểu đáng kể số lượng trường hợp cần xét. Một số bài toán tiêu biểu sử dụng kỹ thuật này bao gồm: bài toán N quân hậu, sinh tổ hợp, sinh hoán vị, bài toán người du lịch (TSP), ...
- **Bitmask** là kỹ thuật sử dụng các phép toán nhị phân để biểu diễn và xử lý trạng thái một cách ngắn gọn và hiệu quả về mặt tính toán. Khi kết hợp với đệ quy hoặc quy hoạch động (Dynamic Programming), bitmask đặc biệt hữu ích trong việc giải quyết các bài toán tổ hợp có không gian trạng thái lớn nhưng có thể mã hóa gọn trong phạm vi số bit của một số nguyên. Kỹ thuật này giúp tối ưu hóa cả về thời gian và bộ nhớ, rất phù hợp với các bài toán yêu cầu hiệu suất cao. Trong chuyên đề này, tôi tập trung vào thao tác bitmask trực tiếp trên số nguyên thay vì sử dụng kiểu dữ liệu bitset, nhằm tối ưu thời gian thực thi và đồng thời giúp học sinh hiểu rõ cách thao tác bit ở cấp độ thấp.

Việc nắm vững hai kỹ thuật trên không chỉ giúp học sinh giải quyết tốt các bài toán khó trong các kỳ thi học sinh giỏi mà còn tạo nền tảng vững chắc để tiếp cận các thuật toán nâng cao như: quy hoạch động theo trạng thái, thuật toán trên đồ thị, cấu trúc dữ liệu cây, tìm kiếm nhánh cận...

Điểm nổi bật của chuyên đề này là việc triển khai **mỗi bài toán bằng cả hai phương pháp: Backtracking và Bitmask**. Thông qua đó so sánh hai cách tiếp cận trên cùng một bài toán, học sinh có cơ hội:

- Rèn luyện kỹ năng lập trình thực tế
- Phát triển tư duy phân tích, đánh giá và so sánh các giải pháp

- Hình thành khả năng lựa chọn chiến lược giải quyết bài toán phù hợp với đặc điểm và yêu cầu cụ thể
- Bước đầu giới thiệu phương pháp bitmask cho học sinh làm quen để học các chuyên đề nâng cao liên quan đến bitmask sau này.

Mỗi bài toán trong chuyên đề đều được trình bày theo một cấu trúc bao gồm:

- Phân tích và định hướng lời giải từng phương pháp
- Mã nguồn minh họa cho từng phương pháp
- Đánh giá độ phức tạp thuật toán của từng phương pháp
- Bảng so sánh ưu – nhược điểm của từng phương pháp
- Bộ test kiểm tra kết quả.

Hy vọng, chuyên đề này sẽ trở thành nguồn tài liệu tham khảo hữu ích, giúp học sinh tự học hiệu quả, củng cố và nâng cao tư duy thuật toán, cũng như phát triển kỹ năng lập trình thi đấu một cách bài bản và chủ động.

2. Một số khái niệm cơ bản

2.1 Kiến thức liên quan

- ❶ Các phép toán trên bit
- ❷ Phương pháp Quay lui (**Backtracking**)
- ❸ Phương pháp **Bitmask**

2.2 Lý thuyết về xử lý bit

2.2.1. Các phép toán trên bit

A	B	$\sim A$	$A \& B$	$A B$	$A \wedge B$	$A \ll k = a * 2^k$	$B \gg k = B / 2^k$
0	0	1	0	0	0	$0011 \ll 4 = 110000 = 48$	$0101 \gg 4 = 0000 = 0$
0	1	1	0	1	1	$0011 \ll 3 = 11000 = 24$	$0101 \gg 3 = 0000 = 0$
1	0	0	0	1	1	$0011 \ll 2 = 1100 = 12$	$0101 \gg 2 = 0001 = 1$
1	1	0	1	1	0	$0011 \ll 1 = 0110 = 6$	$0101 \gg 1 = 0010 = 2$
3	5	12	1	7	6		

Bảng 2.1 – Mô tả các phép toán xử lý trên bit.

2.2.2. Một số thao tác xử lý bit [1]

2.2.2.1 $(n \& (1 \ll i) \neq 0)$ hoặc $(n \gg i) \& 1$ lấy bit thứ i của số nguyên n (0 hay 1)

❖ Cách 1: $(n \& (1 \ll i) \neq 0)$

Ta có thể kiểm tra một bit ở vị trí i của số n có phải bằng 1 hay bằng 0 thông qua các bước sau:

- **Bước 1:** Tạo ra một số mà chỉ có bit thứ i là 1, các bit còn lại là 0 bằng công thức $(1 \ll i)$.
- **Bước 2:** $n \& (1 \ll i)$: phép AND bit giữa n và $1 \ll i$ sẽ cho kết quả khác 0 nếu bit thứ i trong n là 1.
- **Bước 3:** So sánh $\neq 0$ để chuyển thành giá trị bool (true hoặc false).

Nghĩa là:

+ Bit thứ i của n đang bằng 1 đem & với $(1 \ll i) = 1$ thì trả về 1.
+ Bit thứ i của n đang bằng 0 đem & với $(1 \ll i) = 1$ thì trả về 0.

Ví dụ: Số 10 được biểu diễn bằng 8 bit là:

0	0	0	0	1	0	1	0
7	6	5	4	3	2	1	0

$(10 \& (1 \ll 2) = 0 \neq 0 \Rightarrow$ Kết quả là False (0)

$(10 \& (1 \ll 3) = 8 \neq 0 \Rightarrow$ Kết quả là True (1)

❖ Cách 2: $(n \gg i) \& 1$

$(n \gg i)$: Dịch số n sang phải i bit. Lúc này, bit thứ i đã nằm ở vị trí 0

$(n \gg i) \& 1$: Lấy đúng 1 bit thấp nhất (bit thứ 0). Kết quả cuối cùng 0 hoặc 1 — chính là giá trị bit thứ i

2.2.2.2 $n | (1 \ll i)$ bật bit thứ i của số nguyên n (gán giá trị 1 cho bit thứ i):

- **Bước 1:** Tạo số nguyên với bit 1 ở vị trí thứ i , các bit còn lại đều bằng 0 là: $(1 \ll i)$.
- **Bước 2:** Lấy số nguyên n đem OR với $(1 \ll i)$.

Nghĩa là:

+ Bit thứ i của n đang bằng 1 hay 0 gì thì đem | với $(1 \ll i) = 1$ thì cũng trả về 1.

0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---

Ví dụ: 10 được biểu diễn 8 bit là: 7 6 5 4 3 2 1 0

$n = (10 | (1 << 2)) = 14 \rightarrow$ kết quả $n = 14$:

0	0	0	0	1	1	1	0
7	6	5	4	3	2	1	0

2.2.2.3 $n \& (\sim(1 << i))$ hoặc $n - (1 << i)$ tắt bit thứ i của số nguyên n (gán giá trị 0 cho bit thứ i):

❖ **Cách 1: $n \& (\sim(1 << i))$**

- **Bước 1:** Tạo số nguyên với bit 1 ở vị trí thứ i , các bit còn lại đều bằng 0, rồi phủ định nó bằng toán tử NOT: $\sim(1 << i)$.
- **Bước 2:** Lấy số nguyên n đem AND với $\sim(1 << i)$.

Nghĩa là:

+ Bit thứ i của n đang bằng 1 hay 0 gì khi đem & với $\sim(1 << i) = 0$ thì cũng trả về 0.

Ví dụ: 10 được biểu diễn 8 bit là:

0	0	0	0	1	0	1	0
7	6	5	4	3	2	1	0

$n = (10 \& (\sim(1 << 3))) \rightarrow$ kết quả $n = 2$

0	0	0	0	0	0	1	0
7	6	5	4	3	2	1	0

❖ **Cách 2: $n - (1 << i)$** dùng để tắt bit thứ i của số nguyên n nếu nó đang là 1.

- ☐ Nếu bit thứ i trong n đang là 1, thì phép trừ này sẽ tắt nó đi, vì:
- ☐ 1 ở vị trí đó sẽ bị trừ mất, chuyển từ 1 thành 0.
- ☐ Nếu bit thứ i đang là 0, thì phép trừ vẫn thực hiện nhưng kết quả không chính xác - sẽ làm thay đổi các bit khác.

Ví dụ: 10 được biểu diễn 8 bit là:

0	0	0	0	1	0	1	0
7	6	5	4	3	2	1	0

$n = (10 - (1 \ll 3)) \rightarrow$ kết

quả $n = 2$

0	0	0	0	0	0	1	0
7	6	5	4	3	2	1	0

$n = (10 - (1 \ll 2)) \rightarrow$ kết

quả $n = 6$

0	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---

2.2.2.4 $n \wedge (1 \ll i)$ lập bit thứ i của số nguyên n .

Lập bit là thao tác đảo ngược giá trị của bit, từ **0** thành **1** hoặc từ **1** thành **0**. Sử dụng toán tử **XOR**, ta có thể đảo bit ở vị trí i của số n như sau:

- **Bước 1:** Tạo số với bit **1** ở vị trí i bằng công thức $(1 \ll i)$.
- **Bước 2:** Lấy số n đem **XOR** với số $1 \ll i$ để thu được số mới đã lập bit tại vị trí i : $n = n \wedge (1 \ll i)$.

Nghĩa là :

+ bit thứ i đang bằng **1** mà đem xor với $(1 \ll i) = 1$ thì trả về **0**.

+ bit thứ i đang bằng **0** mà đem xor với $(1 \ll i) = 1$ thì trả về **1**.

Ví dụ: 10 được biểu diễn 8 bit là:

0	0	0	0	1	0	1	0
7	6	5	4	3	2	1	0

$n = (10 \wedge ((1 \ll 1))) \rightarrow$ kết quả $n = 8$

0	0	0	0	1	0	0	0
7	6	5	4	3	2	1	0

2.2.2.5 `__builtin_popcount(x)` đếm số lượng bit **1** trong biểu diễn nhị phân của số x .

Ví dụ $= 7 = 111$, thì `__builtin_popcount(7) = 3`

2.2.2.6 `__builtin_ctz(x)` đếm số lượng bit **0** liên tiếp từ phía phải của một số nguyên x , cho đến khi gặp bit **1** đầu tiên.

Ví dụ, $x = 40 = 00101000$. $\Rightarrow$ Kết quả bằng **3**

2.2.2.7 x & -x: Lấy ra bit 1 ở vị trí thấp nhất từ **phải** sang **trái** của một số nguyên x
ví dụ, $x = 88 = 1011000$, thì $\text{bit} = x \& -x = 8 = 2^3$

Vị trí bit	6	5	4	3	2	1	0
x	1	0	1	1	0	0	0
~x	0	1	0	0	1	1	1
-x = ~x + 1	0	1	0	1	0	0	0
x & -x = 8	0	0	0	1	0	0	0
=> vị trí bit phải nhất bằng 1 của 88 là vị trí số 3 từ phải qua							

2.3 Phương pháp Quay Lui (Backtracking) giải các bài toán tổ hợp

2.3.1 Thuật toán Quay Lui [2]

Thuật toán quay lui thường được sử dụng để **liệt kê các cấu hình** của một bài toán. Nguyên tắc hoạt động của thuật toán là **xây dựng nghiệm từng phần tử một trong vector nghiệm**, tại mỗi bước, thử tất cả các khả năng có thể của phần tử đó trước khi chuyển sang phần tử tiếp theo.

Mô tả thuật toán quay lui

- Giả sử bài toán yêu cầu liệt kê tất cả các cấu hình dạng $X = (x_1, x_2, \dots, x_n)$ với $x[i]$ là giá trị được chọn tại vị trí thứ i trong cấu hình.
- **n**: độ dài của cấu hình cần sinh.
- Hàm **Try(i)**: thử giá trị cho vị trí thứ i.

Thuật toán thực hiện như sau:

Bước 1: Kiểm tra đã xây dựng cấu hình cuối cùng chưa: nếu $i == n + 1$, nghĩa là đã xây dựng xong một cấu hình đầy đủ:

- Thực hiện xuất kết quả mong muốn (ví dụ: in ra cấu hình, đếm số nghiệm, lưu kết quả...).
- Kết thúc nhánh đệ quy này bằng **return**.

Ngược lại, thực hiện bắt đầu từ bước 2.

Bước 2: Xét tất cả các giá trị mà x_1 có thể nhận, thử lần lượt từng giá trị đó.

Bước 3: Với mỗi giá trị x_1 đã chọn, tiếp tục xét tất cả các giá trị có thể của x_2 , rồi thử lần lượt từng giá trị đó.

Bước 4: Tiếp tục với $x_3, x_4, \dots, x_n$, mỗi lần chọn một giá trị khả thi, ta thử tiếp giá trị cho phần tử tiếp theo.

Bằng cách này, thuật toán quay lui có thể được mô tả một cách tổng quát như sau:

```
void Try(int i) {
    if (i == n+1) {
```

```
// Nếu đã đạt đến phần tử cuối cùng, in ra nghiệm
Xuất_kết_quả(x);
return;
}
for (mọi giá trị j có thể gán cho x[i]) {
    if (thỏa mãn điều kiện ràng buộc) {
        x[i] = j; // Gán giá trị cho phần tử thứ i
        Try(i + 1); // Gọi đệ quy để tiếp tục với phần tử tiếp theo
        // Nếu cần, quay lui bỏ ghi nhận giá trị x[i] để thử giá trị khác
        x[i] = giá trị ban đầu;
    }
}
}
} // Thuật toán bắt đầu từ Try(1) để thử nghiệm giá trị đầu tiên trong cấu hình
```

2.4 Phương pháp sinh dùng bitmask

2.4.1 Khái niệm về Bitmask

- **Bitmask** là một cách sử dụng một số nguyên để biểu diễn **tập hợp nhị phân**.
- Mỗi **bit** trong biểu diễn nhị phân của số nguyên đại diện cho **một trạng thái**:
 - 0: không chọn phần tử đó
 - 1: chọn phần tử đó

Ví dụ:

Cho tập $S = \{a, b, c\}$ có 3 phần tử.

- Tổng cộng có $2^3 = 8$ tập con.
- Mỗi tập con có thể biểu diễn bởi một **bitmask 3 bit** như sau:

Số thập phân	Bitmask (nhị phân)	Tập con tương ứng
0	000	$\emptyset$
1	001	{c}
2	010	{b}
3	011	{b, c}
4	100	{a}
5	101	{a, c}
6	110	{a, b}
7	111	{a, b, c}

Bảng 2.2 – Số thập phân và nhị phân để biểu diễn tập con tương ứng.

2.4.2 Thuật toán sinh tổ hợp bằng Bitmask

- **Bước 1:** Duyệt tất cả các bitmask từ 0 đến 2^N-1
- **Bước 2:** Với mỗi bitmask, kiểm tra bit thứ i

```
void Bitmask() {
    for (int mask = 0; mask < (1 << N); mask++) {
        // xử lý mask
        for (int i = 0; i < N; i++) {
            if (mask & (1 << i)) {
                // phần tử i được chọn
            }
        }
    }
}
```

3. Một số bài tập áp dụng

3.1 Bài tập 1: Liệt kê dãy nhị phân (0,1) độ dài N.

Nguồn: Giải thuật và lập trình của Lê Minh Hoàng [2]

Cho một số nguyên dương N ($1 \leq N \leq 15$), hãy liệt kê tất cả các dãy nhị phân (chỉ gồm các chữ số 0 và 1) có đúng N chữ số, theo thứ tự tăng dần từ 000...0 đến 111...1.

Dữ liệu vào: Tập văn bản NHIPHAN.INP chứa một dòng duy nhất ghi số nguyên N với ($1 < N \leq 15$)

Dữ liệu ra: Tập văn bản NHIPHAN.OUT gồm 2^N dòng. Mỗi dòng gồm hai phần:

- Chỉ số thứ tự (từ 1 đến 2^N), có dấu chấm (.) phía sau.
- Chuỗi nhị phân độ dài N tương ứng.

Ví dụ:

NHIPHAN.INP	NHIPHAN.OUT
3	1. 000 2. 001 3. 010 4. 011 5. 100 6. 101 7. 110 8. 111

3.1.1 Cách 1: giải bằng phương pháp quay lui

Nghiệm của bài toán được biểu diễn dưới dạng một vector có cấu trúc: $X = [x_1, x_2, \dots, x_n]$, trong đó mỗi phần tử x_i nhận giá trị nhị phân (0 hoặc 1).

Thuật toán được thực hiện như sau:

Xây dựng hàm đệ quy **Try(i)** với mục tiêu là thử gán giá trị cho biến x_i tại vị trí thứ i trong **vector** nghiệm.

Bước 1: Kiểm tra điều kiện dừng của đệ quy.

- Nếu $i == (n+1)$ (tức đã gán giá trị cho toàn bộ các biến từ x_1 đến x_n), thì in ra cấu hình nghiệm hiện tại.

Bước 2: Nếu chưa đạt điều kiện dừng

- Thực hiện thử tất cả các giá trị có thể của x_i là $j \in \{0, 1\}$. Với mỗi giá trị j được gán cho x_i , tiếp tục đệ quy để thử các giá trị cho x_{i+1} thông qua lời gọi đệ quy **Try(i + 1)**. Quá trình tiếp tục cho đến khi tạo được một cấu hình đầy đủ.

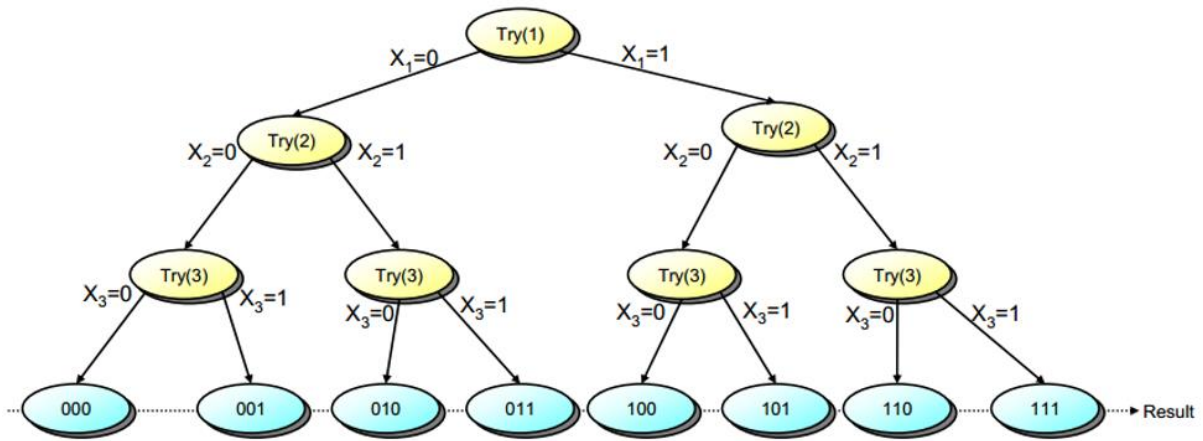
Với mỗi biến nhận hai giá trị nhị phân, số lượng cấu hình nghiệm là 2^n . Ví dụ, với $n = 3$, tổng số cấu hình là $2^3 = 8$.

3.1.2 Chương trình tham khảo cách 1

```
#include <bits/stdc++.h>
using namespace std;
vector<int> X;
int N, dem = 0;
void Try(int i) {
    if (i == N + 1) {
        dem++;
        cout << dem << ". ";
        for (int j = 1; j <= N; j++)
            cout << X[j];
        cout << "\n";
        return;
    }
    for (int j = 0; j <= 1; j++) {
        X[i] = j;
        Try(i + 1);
    }
}
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    freopen("NHIPHAN.INP", "r", stdin);
    freopen("NHIPHAN.OUT", "w", stdout);
    cin >> N;
    X.resize(N + 1, -1);
    Try(1);
    return 0;
}
```

3.1.3 Ví dụ minh họa cách 1

Khi $N = 3$, thì quá trình chạy chương trình được mô phỏng như hình cây tìm kiếm quay lui như hình 3.1.



Hình 3.1 – Cây tìm kiếm quay lui trong bài toán liệt kê dãy nhị phân độ dài N [2].

3.1.4 Độ phức tạp thời gian

- **Số lần gọi đệ quy:** Mỗi phần tử có 2 lựa chọn, nên tổng số nhánh là 2^N .
- **Mỗi lần gọi đệ quy duyệt qua n phần tử** để in dãy, nên độ phức tạp là $O(N \cdot 2^N)$.
- **Không gian nhớ:** Do sử dụng đệ quy nên cần $O(N)$ bộ nhớ cho stack.

3.1.5 Cách 2: giải bằng phương pháp khử đệ quy dùng bitmask

- Có 2^N dãy nhị phân độ dài N.
- Với mỗi số nguyên i từ 0 đến $2^N - 1$, biểu diễn nhị phân của số nguyên i chính là một dãy nhị phân độ dài N.
- Dùng phép dịch phải ($>>$) để tách từng bit từ cao đến thấp: $(i >> j) \& 1$ rồi in nó ra.

3.1.6 Chương trình tham khảo cách 2

```

#include <bits/stdc++.h>
using namespace std;
int N;
void Bitmask(int N) {
    for (int mask = 0; mask < (1<<N); mask++) {
        cout << mask + 1 << ". ";
        for (int i = N - 1; i >= 0; i--)
            cout << ((mask >> i) & 1);
        cout << "\n";
    }
}
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    freopen("NHIPHAN.INP", "r", stdin);
    freopen("NHIPHAN.OUT", "w", stdout);
    cin >> N;
    Bitmask(N);
    return 0;
}
    
```

3.1.7 Ví dụ mô phỏng cách 2

- Với $N = 3$ Có tổng cộng $2^3 = 8$ dãy nhị phân độ dài 3.
- Mỗi dãy được sinh bằng cách duyệt i từ 0 đến $7=2^3-1$.
- Với mỗi i , in ra bit từ trái qua phải (từ bit cao nhất đến thấp nhất).

i	Nhị phân (3 bit)	Giải thích từng bước ($i \gg j$) & 1
0	000	$0 \gg 2 = 0, 0 \gg 1 = 0, 0 \gg 0 = 0$
1	001	0 0 1
2	010	0 1 0
3	011	0 1 1
4	100	1 0 0
5	101	1 0 1
6	110	1 1 0
7	111	1 1 1

Bảng 3.1 - Mô phỏng chạy thuật toán liệt kê dãy nhị phân bằng bitmask.

3.1.8 Độ phức tạp thời gian

- **Số vòng lặp ngoài:** Chạy từ 0 đến $2^N - 1$, tức là $O(2^N)$.
- **Số vòng lặp trong:** Chạy N lần để lấy từng bit,
- Tổng độ phức tạp là $O(N * 2^N)$.
- **Không gian nhớ:** Chỉ dùng một số biến đơn giản nên $O(1)$.

3.1.9 So sánh hai thuật toán

Tiêu chí	Quay lui (Try())	Bitmask (Bitmask())
Độ phức tạp thời gian	$O(N * 2^N)$	$O(N * 2^N)$
Độ phức tạp không gian	$O(N)$ (stack đệ quy)	$O(1)$ (vài biến đơn giản)
Ưu điểm	Dễ hiểu, trực quan, phù hợp với	Chương trình ngắn gọn, đơn

	những người mới học giải thuật. Dễ dàng hình dung quá trình tìm kiếm lời giải thông qua cây trạng thái.	giản: Khi quen với thao tác bit, ta có thể viết các chương trình rất tối giản, tránh đệ quy.
Nhược điểm	Chậm nếu N lớn. Do đệ quy sâu và số lượng trạng thái tăng nhanh, phương pháp này có thể bị tràn stack.	Cần hiểu rõ thao tác bit. Với người mới học, việc dùng toán tử bit (AND, OR, XOR, shift...) có thể khó hiểu và dễ sai sót.
⇒ Cần hiểu rõ hai phương pháp tạo nền tảng vững chắc để tiếp cận các thuật toán nâng cao và quen dần với thao tác bit. ⇒ Hai thuật toán hoạt động hiệu quả tương đương nhau.		

Bảng 3.2 – So sánh hai thuật toán quay lui và bitmask trong bài liệt kê dãy nhị phân.

3.1.10 Test

https://drive.google.com/file/d/1XscvilkDk_SX8V6fLYa56U0GUi2GiIb/view?usp=sharing

3.2 Bài tập 2: Liệt kê tổ hợp

Nguồn: Giải thuật và lập trình của Lê Minh Hoàng [2]

Cho một tập hợp gồm N phần tử nguyên dương, được đánh số liên tiếp bắt đầu từ 1, tức là: $A = \{1, 2, 3, \dots, N\}$. Hãy liệt kê tất cả các **tập con gồm đúng k phần tử** của tập A .

Lưu ý: Các tập con chỉ khác nhau về thứ tự (tức là hoán vị của nhau) chỉ được tính là **một tập con duy nhất**.

Dữ liệu vào: Tập văn bản **TOHOP.INP** chứa một dòng duy nhất ghi hai số nguyên N và K ($1 \leq K \leq N \leq 20$)

Dữ liệu ra: Tập văn bản **TOHOP.OUT** gồm:

- Ghi ra tất cả các tập con có đúng K phần tử của tập A , mỗi tập con trên một dòng.
- Các phần tử trong mỗi tập con phải được in theo thứ tự tăng dần.
- Trước mỗi dòng ghi số thứ tự của tập con, theo định dạng: **i. a1 a2 a3 ... ak**

Ví dụ:

TOHOP.INP	TOHOP.OUT
4 3	1. 1 2 3 2. 1 2 4 3. 1 3 4 4. 2 3 4

3.2.1 Cách 1: giải bằng phương pháp quay lui

Đề liệt kê tất cả các tập con có đúng **k phần tử** của tập hợp $A = \{1, 2, \dots, n\}$. Mỗi tập con **k** phần tử có thể được xem như một dãy tăng dần: $(x_1, x_2, \dots, x_k)$ sao cho:

- Mỗi x_i là một phần tử trong A
- Và $x_1 < x_2 < \dots < x_k$

Vì các phần tử phải khác nhau và tăng dần, nên:

- $x_k \leq n$
- $x_{k-1} \leq x_k - 1 \leq n - 1$
- ...
- $x_i \leq n - k + i$

Ví dụ: nếu $n = 4, k = 3$, thì:

- $x_1 \leq 2$
- $x_2 \leq 3$
- $x_3 \leq 4$

Để chọn hợp lệ, mỗi phần tử x_i (với i từ 1 đến k) cần thoả mãn thì:

$x_i \in [x_{i-1} + 1, n - k + i]$ (Ta giả sử thêm $x_0 = 0$ để bắt đầu cho $i = 1$)

Xây dựng hàm đệ quy **Try(i)** với mục tiêu là thử gán giá trị cho biến x_i tại vị trí thứ i trong vector nghiệm $X = [x_1, x_2, \dots, x_k]$.

Thuật toán đệ quy và quay lui được thực hiện như sau:

Bước 1: Kiểm tra điều kiện dừng của đệ quy.

- Nếu $i == (k+1)$ (tức đã gán giá trị cho toàn bộ các biến từ x_1 đến x_k), thì in ra cấu hình nghiệm hiện tại.

Bước 2: Nếu chưa đạt điều kiện dừng.

Thực hiện thử tất cả các giá trị có thể của x_i là $j \in [X[i-1] + 1, n-k+1]$. Với mỗi giá trị j được gán cho x_i , tiếp tục đệ quy để thử các giá trị cho x_{i+1} thông qua lời gọi đệ quy **Try(i + 1)**. Quá trình tiếp tục cho đến khi tạo được một cấu hình đầy đủ.

- Bắt đầu với x_1 , những giá trị j để thử gán cho x_1 là từ 1 đến $n - k + 1$
- Với mỗi x_1 đã nhận, chọn x_2 từ $x_1 + 1$ đến $n - k + 2$
- Với mỗi x_2 đã nhận, chọn x_3 từ $x_2 + 1$ đến $n - k + 3$
- Cứ tiếp tục như vậy đến x_k

Mỗi lần chọn đủ **k giá trị hợp lệ**, ta được một tập con cần liệt kê.

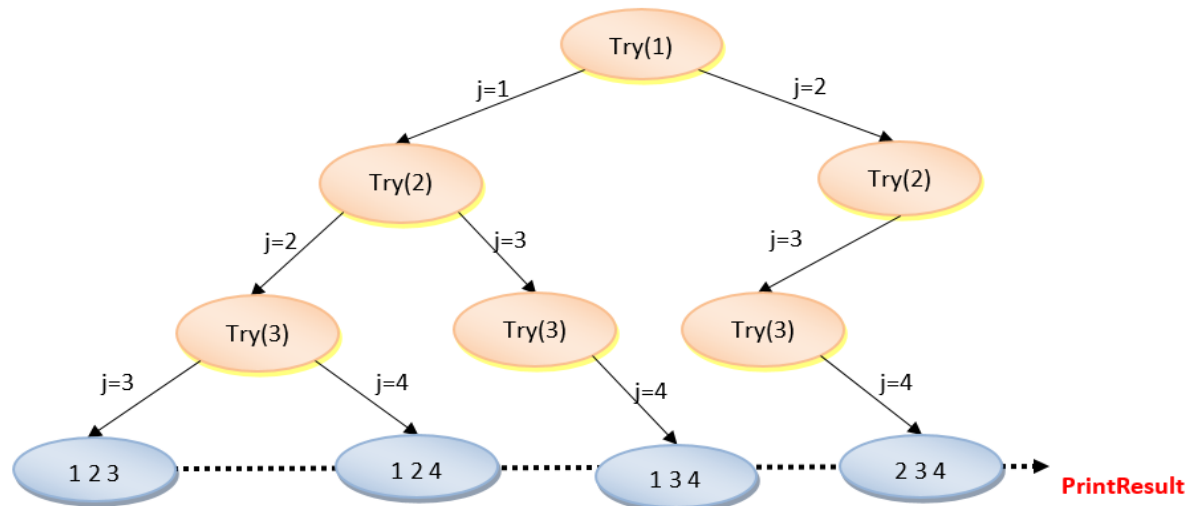
$$\text{Số cấu hình } C_N^K = \frac{N!}{K!(N-K)!} = \frac{4!}{3!(4-3)!} = 4$$

3.2.2 Chương trình tham khảo cách 1

```
#include <bits/stdc++.h>
using namespace std;
vector <int> X;
int N, K, dem=0;
void Try(int i){
    if (i==K+1) {
        dem++;
        cout<<dem<<" ";
        for (int j=1; j<=K; j++) cout<<X[j]<<" ";
        cout<<"\n";
        return;
    }
    for(int j=X[i-1]+1; j<=N-K+i; j++) {
        X[i]=j;
        Try(i+1);
    }
}
int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("TOHOP.INP", "r", stdin);
    freopen("TOHOP.OUT", "w", stdout);
    cin>>N>>K;
    X.resize(K+1);
    X[0]=0;
    Try(1);
    return 0;
}
```

3.2.3 Ví dụ minh họa cách 1

Khi $N = 4, K = 3$ thì quá trình chạy chương trình được mô phỏng như hình cây tìm kiếm quay lui như hình 3.2.



Hình 3.2 - Cây tìm kiếm quay lui trong bài liệt kê tập con k phần tử của tập n phần tử.

3.2.4 Độ phức tạp thời gian cách 1

- Số lần gọi đệ quy: $C(N, K)$

- Chi phí mỗi lần gọi: $O(K)$
- Tổng độ phức tạp: $O(K \times C(N, K))$
- Không gian nhớ: $O(N)$ (do dùng mảng + đệ quy sâu đến mức thứ K)

3.2.5 Cách 2: giải bằng phương pháp khử đệ quy dùng bitmask

- Mỗi tập con của $\{1, 2, \dots, N\}$ có thể biểu diễn bằng một số nguyên từ 0 đến $2^N - 1$, trong đó:
- Bit thứ i (tính từ phải sang trái, bắt đầu từ 0) được bật (1) nghĩa là phần tử $i + 1$ có trong tập con.
- Để liệt kê tất cả **tập con có đúng K phần tử**, ta duyệt tất cả các số từ 0 đến $2^N - 1$ và chỉ in ra các tập con có đúng K bit 1 (tức là đúng K phần tử).

3.2.6 Chương trình tham khảo cách 2

```
#include <bits/stdc++.h>
using namespace std;
int N, K, dem = 0;
int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("input.inp", "r", stdin);
    freopen("output.out", "w", stdout);

    cin >> N >> K;
    for (int mask = 0; mask < (1<<N); mask++) {
        if (__builtin_popcount(mask) == K) {
            dem++;
            cout << dem << ". ";
            for (int i = 0; i < N; i++) {
                if (mask & (1 << i)) {
                    cout << (i + 1) << " ";
                }
            }
            cout << "\n";
        }
    }
    return 0;
}
```

3.2.6 Ví dụ mô phỏng cách 2

Bitmask từ 0 đến $2^4 - 1 = 15$ (tức 0000 đến 1111) chỉ chọn những mask có đúng **3 bit 1**

mask (nhị phân)	Bit bật	Tập con	Số bit 1	In ra không?
0000 (0)	-	{}	0	Không
0001 (1)	0	{1}	1	Không
0010 (2)	1	{2}	1	Không
0011 (3)	0,1	{1,2}	2	Không
0100 (4)	2	{3}	1	Không
0101 (5)	0,2	{1,3}	2	Không
0110 (6)	1,2	{2,3}	2	Không

0111 (7)	0,1,2	{1,2,3}	3	Có
1000 (8)	3	{4}	1	Không
1001 (9)	0,3	{1,4}	2	Không
1010 (10)	1,3	{2,4}	2	Không
1011 (11)	0,1,3	{1,2,4}	3	Có
1100 (12)	2,3	{3,4}	2	Không
1101 (13)	0,2,3	{1,3,4}	3	Có
1110 (14)	1,2,3	{2,3,4}	3	Có
1111 (15)	0,1,2,3	{1,2,3,4}	4	Không

Bảng 3.3 - Mô phỏng chạy thuật toán liệt kê tập con k phần tử trong tập N phần tử bằng bitmask.

3.2.8 Độ phức tạp thời gian cách 2

- Số lần duyệt (mask): 2^N
- Mỗi lần in tổ hợp: $O(N)$
- Tổng độ phức tạp: $O(N \times 2^N)$
- Không gian nhớ: $O(1)$ (chỉ dùng biến tạm)

3.2.9 So sánh hai thuật toán

mask (nhị phân)	Quay lui	Bitmask
Độ phức tạp thời gian	$O(K \times C(N, K))$ (hiệu quả hơn)	$O(N \times 2^N)$ (duyệt toàn bộ tập con)
Độ phức tạp không gian	$O(N)$ (do mảng và stack đệ quy)	$O(1)$ (chỉ dùng biến đếm, mask, vòng lặp)
Ưu điểm	Tối ưu hơn nếu chỉ sinh đúng tổ hợp cần, dễ thêm điều kiện ràng buộc	Code ngắn, không dùng đệ quy, dễ cài nhanh
Nhược điểm	Cần đệ quy, dễ gây tràn stack nếu không kiểm soát	Chậm khi N lớn, phải duyệt cả 2^N cấu hình
⇒ Thuật toán quay lui hoạt động tốt hơn		

Bảng 3.4 – So sánh hai thuật toán quay lui và bitmask trong bài liệt kê tổ hợp.

3.2.10 Test

https://drive.google.com/file/d/1_s_u20KyHaH405f7MEySAzZDckyK-itI/view?usp=sharing

3.3 Bài tập 3: Liệt kê chỉnh hợp

Nguồn: Giải thuật và lập trình của Lê Minh Hoàng [2]

Cho một tập hợp S gồm N phần tử nguyên dương liên tiếp $S = \{1, 2, \dots, N\}$. Hãy liệt kê tất cả các tập con gồm đúng K phần tử được chọn từ tập S , sao cho các phần tử trong mỗi tập con đều khác nhau từng đôi một.

Dữ liệu vào: Tập văn bản **CHINHHOP.INP** chứa một dòng duy nhất ghi hai số nguyên N ($1 \leq N \leq 20$) và K ($2 \leq K \leq 5$)

Dữ liệu ra: Tập văn bản CHINHHOP.OUT gồm:

- Tất cả các tập con có đúng **K** phần tử khác nhau từng đôi một, mỗi tập con trên một dòng.
- Mỗi dòng bắt đầu bằng số thứ tự của tập con, theo định dạng: **i. a1 a2 ... ak**
- Các tập con được liệt kê theo thứ tự từ điển tăng dần.

Ví dụ:

CHINHHOP.INP	CHINHHOP.OUT
3 2	1. 1 2 2. 1 3 3. 2 1 4. 2 3 5. 3 1 6. 3 2

3.3.1 Cách 1: giải bằng phương pháp quay lui

Để liệt kê tất cả các chỉnh hợp không lặp chập **k** của tập hợp $S=\{1,2,...,n\}$, ta có thể biểu diễn mỗi chỉnh hợp dưới dạng một cấu hình $X= [x_1, x_2, ..., x_k]$, trong đó các phần tử $x_i \in S$ và đôi một khác nhau.

Thủ tục **Try(i)** - xét tất cả các khả năng chọn x_i - sẽ thử hết các giá trị từ **1 đến n**, mà các giá trị này chưa bị các phần tử đứng trước chọn.

Để kiểm soát việc sử dụng các phần tử, ta sử dụng kỹ thuật **mảng đánh dấu**. Cụ thể, ta khai báo một mảng logic:

Free[1..n]

trong đó:

- **Free[j]=false**: phần tử **j** hiện đang **tự do**, có thể chọn.
- **Free[j]=true**: phần tử **j** đã **được sử dụng** ở một vị trí trước đó.

Bước khởi tạo: tất cả các phần tử của mảng **Free** được gán giá trị **false**, nghĩa là ban đầu, tất cả các phần tử trong **S** đều tự do.

Thuật toán được thực hiện như sau:

Xây dựng hàm đệ quy **Try(i)** với mục tiêu là thử gán giá trị cho biến x_i tại vị trí thứ **i** trong **vector** nghiệm $X = [x_1, x_2, ..., x_k]$.

Bước 1: Kiểm tra điều kiện dừng của đệ quy.

- Nếu **i == (k+1)** (tức đã gán giá trị cho toàn bộ các biến từ x_1 đến x_k), thì in ra cấu hình nghiệm hiện tại.

Bước 2: Nếu chưa đạt điều kiện dừng

- Duyệt qua tất cả các giá trị $j \in \{1, \dots, n\}$ thỏa mãn $\text{Free}[j] = \text{false}$
- Gán $x_i = j$, sau đó đánh dấu j là đã được sử dụng: $\text{Free}[j] = \text{true}$
- Gọi đệ quy để tiếp tục chọn giá trị cho x_{i+1} : $\text{Try}(i + 1)$.
- Sau khi gọi đệ quy thực hiện quay lui để **trả lại** trạng thái ban đầu cho giá trị $\text{Free}[j]$ bằng cách đặt lại $\text{Free}[j] = \text{false}$, cho phép các vị trí sau có thể sử dụng lại giá trị j .

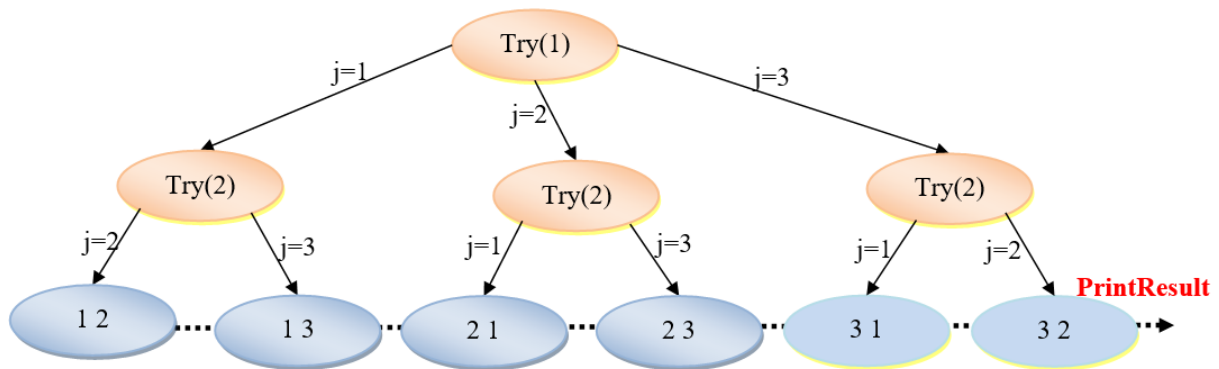
Số cấu hình $P_N^k = \frac{N!}{(N-k)!} = \frac{3!}{(3-2)!} = 6$, nếu $k=N$ thì ta có bài toán liệt kê hoán vị N phần tử.

3.3.2 Chương trình tham khảo cách 1

```
#include <bits/stdc++.h>
using namespace std;
vector<int> X;
vector<bool> Free;
int N, K, dem=0;
void Try(int i){
    if (i==K+1) {
        dem++;
        cout<<dem<<". ";
        for (int i=1; i<=K; i++) cout<<X[i]<<" ";
        cout<<"\n";
        return;
    }
    for (int j=1; j<=N; j++) {
        if (Free[j]==false) {
            X[i]=j;
            Free[j]=true;
            Try(i+1);
            Free[j]=false;
        }
    }
}
int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("input.inp", "r", stdin);
    freopen("output.out", "w", stdout);
    cin>>N>>K;
    X.resize(N+1);
    Free.assign(N+1, false);
    Try(1);
    return 0;
}
```

3.3.3 Ví dụ minh họa cách 1

Khi $N = 3$, $K = 2$ thì quá trình chạy chương trình được mô phỏng như hình cây tìm kiếm quay lui như sau:



Hình 3.3 - Cây tìm kiếm quay lui trong bài Liệt kê các chỉnh hợp không lặp chập k phần tử của tập N phần tử.

3.3.4 Độ phức tạp thời gian cách 1

- Số lần gọi đệ quy: $P(N, K) = \frac{N!}{(N-K)!}$
- Chi phí mỗi lần gọi: $O(K)$
- Tổng độ phức tạp: $O(\frac{N!}{(N-K)!} \times K)$
- Không gian bộ nhớ: $O(N+K)$

3.3.5 Cách 2: giải bằng phương pháp khử đệ quy dùng bitmask

- Duyệt tất cả các số nguyên từ 0 đến $2^N - 1$.
- Mỗi số nguyên **mask** được xem như là **một** tổ hợp con: Bit **i** trong **mask** bật (1) nếu phần tử **i + 1** được chọn.
- Sử dụng hàm **__builtin_popcount(mask)** để chọn ra **chỉ những bitmask có đúng K bit bật** → tương ứng với chọn tổ hợp **K** phần tử.
- Với mỗi bitmask hợp lệ (có K bit bật), **tạo danh sách phần tử được chọn** từ 1 đến N.
- Đảm bảo bắt đầu từ cấu hình nhỏ nhất bằng cách dùng hàm **sort()**.
- Dùng hàm **next_permutation** để liệt kê **tất cả các hoán vị** của tổ hợp hiện tại.

3.3.6 Chương trình tham khảo cách 2

```

#include <bits/stdc++.h>
using namespace std;
int N, K, dem = 0;
int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("input.inp", "r", stdin);
    freopen("output.out", "w", stdout);
    cin >> N >> K;
    vector<int> X;

    for (int mask = 0; mask < (1 << N); ++mask) {
        if (__builtin_popcount(mask) == K) {
            X.clear();
            for (int i = 0; i < N; ++i) {
                if (mask & (1 << i)) {

```

```

        X.push_back(i + 1);
    }
}
sort(X.begin(), X.end());
do {
    dem++;
    cout << dem << ". ";
    for (int x : X) cout << x << " ";
    cout << "\n";
} while (next_permutation(X.begin(), X.end()));
}
return 0;
}

```

3.3.7 Ví dụ mô phỏng cách 2

Với $N = 3, K = 2$. Bitmask từ 0 đến $2^3 - 1 = 7$ (tức 000 đến 111) chỉ chọn những mask có đúng 2 bit 1, sau đó tạo hoán vị của các mask bằng hàm `next_permutation()`

mask (nhị phân)	Bit bật	Tập con	Số bit 1	In ra không?
000 (0)	–	{}	0	Không
001 (1)	1	{1}	1	Không
010 (2)	2	{2}	1	Không
011 (3)	1, 2	{1, 2}	2	Tạo hoán vị {1,2}, {2,1}
100 (4)	3	{3}	1	Không
101 (5)	1, 3	{1, 3}	2	Tạo hoán vị {1,3}, {3,1}
110 (6)	2, 3	{2, 3}	2	Tạo hoán vị {2,3}, {3,2}
111 (7)	1, 2, 3	{1, 2, 3}	3	Không

⇒ Với cách này tuy đưa ra đầy đủ các tập con nhưng không theo thứ tự đề bài. Nên khi tạo trình chấm thì thiết lập trường hợp đa nghiệm hoặc đưa ra số lượng tập con.

Bảng 3.5 – Mô phỏng chạy thuật toán liệt kê chỉnh hợp bằng bitmask.

3.3.8 Độ phức tạp thời gian

- Số lượng tổ hợp K phần tử: $\frac{N!}{K!(N-K)!}$
- Số hoán vị của mỗi tổ hợp: $K!$
- Tổng số tổ hợp – hoán vị cần xử lý: $\frac{N!}{(N-K)!}$
- Chi phí mỗi lần in ra: $O(K)$
- Tổng độ phức tạp thời gian: $O(\frac{N!}{(N-K)!} \times K)$
- Không gian bộ nhớ: $O(K)$

3.3.9 So sánh hai thuật toán

mask (nhị phân)	Quay lui	Bitmask
Độ phức tạp thời gian	$O(\frac{N}{K} \times K! \times K)$	$O(\frac{N}{K} \times K! \times K)$

Độ phức tạp không gian	O(N)	O(K)
Ưu điểm	- Dễ hiểu với người quen tư duy đệ quy - Dễ thêm ràng buộc (ràng buộc vị trí, giá trị...)	- Code ngắn, không dùng đệ quy
Nhược điểm	- Cần đệ quy, dễ gây tràn stack nếu không kiểm soát.	- Hạn chế – khó thêm điều kiện đặc biệt khi sinh hoán vị
⇒ Hai thuật toán hoạt động hiệu quả tương đương nhau, nhưng có kết quả thứ tự xuất hiện các tập con không giống nhau ở kết quả ra.		

Bảng 3.6 - So sánh hai thuật toán quay lui và bitmask trong bài toán liệt kê chỉnh hợp.

3.3.10 Test

* Test chấm kết quả theo cách 1

<https://drive.google.com/file/d/1bMO2CwcUr8AJ7i0Gir3Pw5eehUYmjJ6B/view?usp=sharing>

* Test chấm kết quả theo cách 2

https://drive.google.com/file/d/1dc5sU5BF4SmIeEU9HmxrfKy_HcgHzqLk/view?usp=sharing

3.4 Bài tập 4: Bài toán máy rút tiền tự động ATM.

Nguồn: Tài liệu giáo khoa chuyển tin quyển 1 của Hồ Sĩ Đàm [3]

Một máy ATM hiện có N tờ tiền với mệnh giá lần lượt là $t_1, t_2, \dots, t_n$.

Hãy tìm **một cách** chọn ra một số tờ tiền (không nhất thiết phải chọn hết) sao cho tổng số tiền bằng đúng S. Nếu có cách chọn in **1**. Nếu không thể chọn, in ra **-1**.

Dữ liệu vào: Tập văn bản **ATM.INP** gồm:

- Dòng đầu tiên chứa hai số nguyên N (số lượng tờ tiền) $1 \leq N \leq 20$ và S (số tiền cần đạt được) $0 \leq S \leq 10^6$.
- Dòng thứ hai chứa N số nguyên tương ứng với mệnh giá của từng tờ tiền.

Dữ liệu ra: Tập văn bản **ATM.OUT** gồm một số duy nhất là 1 nếu có cách chọn, -1 nếu không có cách chọn.

INPUT.INPUT	OUTPUT.OUT	Giải thích
10 390 200 10 20 20 50 50 50 50 100 100	1	20 20 50 50 50 100 100
4 150 10 20 50 100	1	50 100

4 25 10 20 30 40	-1	Không có cách chọn nào để đạt được tổng 25
---------------------	----	--

3.4.1 Cách 1: giải bằng phương pháp quay lui

Nghiệm của bài toán được biểu diễn dưới dạng một dãy nhị phân độ dài N , lưu trong vector $X=(X_1, X_2, \dots, X_N)$, trong đó:

- $X[i] = 1$: nếu tờ tiền thứ i được chọn để trả.
- $X[i] = 0$: nếu tờ tiền thứ i không được chọn.

Một cấu hình X là **nghiệm hợp lệ** nếu thỏa mãn:

$$X_1 \times T_1 + X_2 \times T_2 + \dots + X_N \times T_N = S$$

- Gọi **Sum** là giá trị tổng đang có khi xét việc chọn hay không chọn tờ tiền từ tờ tiền thứ 1 đến tờ tiền thứ i .
- S là giá trị tổng cần tìm.

Xây dựng hàm đệ quy **Try(i)** với mục tiêu là thử gán giá trị cho biến x_i tại vị trí thứ i trong vector nghiệm.

Bước 1: Kiểm tra điều kiện dừng và cắt nhánh trước khi thực hiện xây dựng cấu hình $X[i]$.

- Nếu **Sum>S**: không thể tạo ra tổng bằng S từ nhánh hiện tại $\Rightarrow$ **dừng nhánh**.
- Nếu đã xét hết N tờ tiền mà chưa đạt tổng $S \Rightarrow$ **trả về** để thử nhánh khác.
- Nếu **Sum==S**: ta đã tìm được nghiệm $\Rightarrow$ **in kết quả**.

Bước 2: Xây dựng cấu hình $X[i]$ với i từ 1 đến N , tại mỗi bước, có hai lựa chọn:

- Với các giá trị $j \in \{0,1\}$ để thử gán cho $X[i]$ thực hiện gán $X[i]=j$, nghĩa là
 - + Không chọn tờ tiền thứ i : gán $X[i]=0$
 - + Chọn tờ tiền thứ i : gán $X[i]=1$
- Sau đó cập nhật tổng đang tính: **Sum=Sum+X[i]xT[i]**
- Gọi đệ quy **Try(i+1)** để xây dựng cấu hình tiếp theo.

Bước 3: Sau mỗi lời gọi đệ quy, **quay lui** để trả lại giá trị ban đầu cho giá trị tổng đang tính (Sum): **Sum=Sum-X[i]**

3.4.2 Chương trình tham khảo cách 1

```
#include <bits/stdc++.h>
using namespace std;
vector <int> T, X;
int N, S, Sum, dem=0;
```

```

void Try(int i){
    if (i>N+1 || Sum>S) return;
    if (Sum==S) {
        dem++;
        /*cout<<dem<<" ";
        for (int i=1;i<=N;i++)
            if (X[i]==1) cout<<T[i]<<" ";
        cout<<" \n";*/
        return;
    }
    for (int j=0;j<=1;j++){
        X[i]=j;
        Sum=Sum+T[i]*X[i];
        Try(i+1);
        Sum=Sum-T[i]*X[i];
        X[i]=X[i]-j;
    }
}

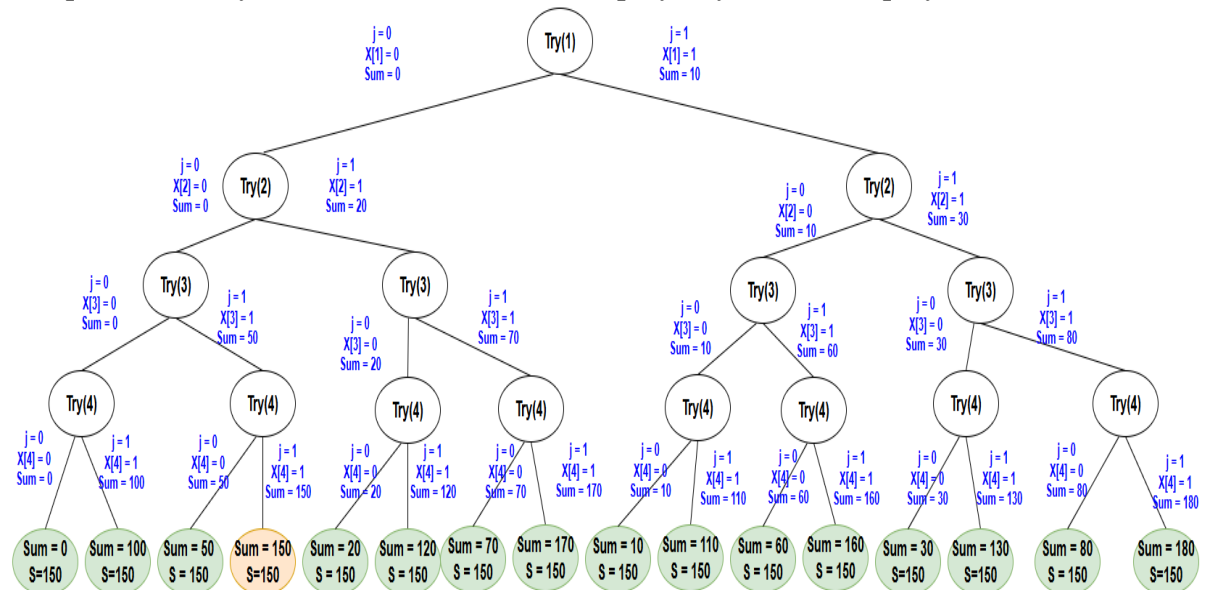
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("input.inp", "r", stdin);
    freopen("output.out", "w", stdout);
    cin>>N>>S;
    X.resize(N+1);
    T.resize(N+1);
    for (int i=1;i<=N;i++) cin>>T[i];
    Sum=0;
    Try(1);
    if (dem==0) cout<<-1;
    else cout<<1;
    return 0;
}

```

3.4.3 Ví dụ minh họa cách 1

Với $N = 4$, $S = 150$, và mệnh giá của các tờ tiền lần lượt là: **10 20 50 100**

Thì quá trình chạy thuật toán được thể hiện quay cây tìm kiếm quay lui như hình 2.4.



Hình 3.4 - Cây tìm kiếm quay lui trong bài máy rút tiền tự động ATM.

3.4.4 Độ phức tạp thời gian

- Số lần gọi đệ quy: 2^N
- Chi phí mỗi lần gọi: $O(N)$
- Tổng độ phức tạp: $O(2^N \cdot N)$
- Không gian bộ nhớ: $O(N)$

3.4.5 Cách 2: giải bằng phương pháp khử đệ quy dùng bitmask

Bitmask là một cách biểu diễn **tập con** của một tập hợp bằng **số nhị phân**.

- Với một mảng có N phần tử, có tất cả 2^N tập con.
- Mỗi tập con có thể được biểu diễn bằng một số nhị phân có N bit (từ 0 đến $2^N - 1$).
 - Bit thứ $i = 1$ nếu chọn phần tử thứ i trong mảng.
 - Bit thứ $i = 0$ nếu **không** chọn phần tử thứ i .

Thuật toán được thực hiện như sau:

Bước 1: Duyệt qua tất cả tập con (từ 0 đến $2^N - 1$)

- Gọi mỗi tập con là một **mask** (giá trị nguyên).
- Với mỗi **mask**, duyệt từng bit:
 - Nếu bit thứ i là 1 , ta cộng $T[i]$ vào tổng đang tính sum.

Bước 2: Kiểm tra tổng của tập con

- Nếu **sum == S** thì tăng biến đếm **dem** lên.

Bước 3: Xuất kết quả

- Nếu không có tập con nào thỏa mãn (**dem == 0**) thì in **-1**
- Ngược lại, in **1** (như yêu cầu của bài toán).

3.4.6 Chương trình tham khảo cách 2

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    freopen("ATM.INP", "r", stdin);
    freopen("ATM.OUT", "w", stdout);

    int N, S;
    cin >> N >> S;
    vector<int> T(N);
    for (int i = 0; i < N; ++i) {
        cin >> T[i];
    }
    int dem = 0;
```

```

for (int mask = 0; mask < (1<<N)-1; ++mask) {
    int sum = 0;
    for (int i = 0; i < N; ++i) {
        if (mask & (1 << i)) {
            sum += T[i];
        }
    }
    if (sum == S) {
        ++dem;
        /*cout << dem << ". ";
        for (int i = 0; i < N; ++i) {
            if (mask & (1 << i)) cout << T[i] << " ";
        }
        cout << "\n";
        */
    }
}

if (dem == 0) cout << -1;
else cout << 1;
return 0;
}

```

3.4.7 Ví dụ mô phỏng

- $N = 5, S = 150$
- Dãy $T = [10, 20, 50, 100]$
- Số tập con là $2^N = 2^4 = 16$
- Duyệt từ $\text{mask} = 0$ đến 15 (0000 đến 1111)

mask (nhị phân)	Tập con được chọn	Tổng	Có đúng $S=150$?
0000 (0)	$\emptyset$	0	Không
0001 (1)	{10}	10	Không
0010 (2)	{20}	20	Không
0011 (3)	{10, 20}	30	Không
0100 (4)	{50}	50	Không
0101 (5)	{10, 50}	60	Không
0110 (6)	{20, 50}	70	Không
0111 (7)	{10, 20, 50}	80	Không
1000 (8)	{100}	100	Không
1001 (9)	{10, 100}	110	Không
1010 (10)	{20, 100}	120	Không
1011 (11)	{10, 20, 100}	130	Không
1100 (12)	{50, 100}	150	Có
1101 (13)	{10, 50, 100}	160	Không
1110 (14)	{20, 50, 100}	170	Không

Bảng 3.7 - Mô phỏng chạy thuật toán máy rút tiền tự động ATM bằng bitmask.

3.4.8 Độ phức tạp thời gian

- Số lần gọi đệ quy: 2^N
- Chi phí mỗi lần gọi: $O(N)$
- Tổng độ phức tạp: $O(2^N \cdot N)$
- Không gian bộ nhớ: $O(N)$

3.4.9 So sánh hai thuật toán

Thành phần	Thuật toán 1 (đệ quy)	Thuật toán 2 (bitmask)
Số lần gọi	2^N nhánh đệ quy	2^N lần lặp qua các bitmask
Chi phí mỗi lần gọi	$O(N)$ (in kết quả khi hợp lệ)	$O(N)$ (duyệt các bit để tính tổng)
Tổng độ phức tạp	$O(2^N * N)$	$O(2^N * N)$
Không gian bộ nhớ	$O(N)$ (mảng T, X)	$O(N)$ (mảng T)
Hiệu suất thực tế	Phức tạp hơn vì có đệ quy	Đơn giản hơn, không cần đệ quy
⇒ Hai thuật toán hoạt động hiệu quả tương đương nhau.		

Bảng 3.8 - So sánh hai thuật toán quay lui và bitmask trong bài toán máy rút tiền tự động ATM.

3.4.10 Test

<https://drive.google.com/file/d/1fO62RmzXaZzqA8d4YUTwapxFselZYEEdn/view?usp=sharing>

3.5 Bài tập 5: Xếp N quân hậu

Nguồn: Giải thuật và lập trình của Lê Minh Hoàng [2]

Xét bàn cờ tổng quát kích thước $n \times n$. ($3 \leq n \leq 12$) Một quân hậu trên bàn cờ có thể ăn được các quân khác nằm tại các ô cùng hàng, cùng cột hoặc cùng đường chéo.

Yêu cầu: Hãy tìm cách xếp n quân hậu trên bàn cờ sao cho không quân nào ăn quân nào.

Dữ liệu vào: Tập văn bản QUANHAU.INP gồm: Một số nguyên duy nhất n ($3 \leq n \leq 12$).

Dữ liệu ra: Tập văn bản QUANHAU.OUT gồm một số duy nhất là số cách đặt n quân hậu hoặc 0 nếu không có cách đặt nào.

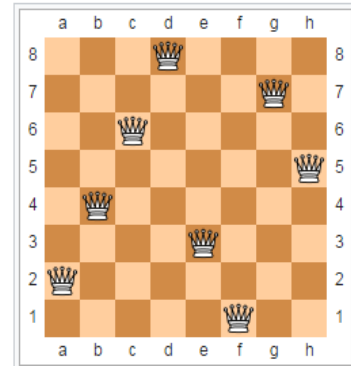
Ví dụ, với $n = 8$ bài toán có đến 92 nghiệm.

QUANHAU.INP	QUANHAU.OUT
8	92

Một cách đặt hậu như hình vẽ:

[6, 1, 5, 2, 8, 3, 7, 4]

- Hậu 1 nằm ở hàng 1 cột 6
- Hậu 2 nằm ở hàng 2 cột 1
- Hậu 3 nằm ở hàng 3 cột 5
- Hậu 4 nằm ở hàng 4 cột 2
- Hậu 5 nằm ở hàng 5 cột 8
- Hậu 6 nằm ở hàng 6 cột 3
- Hậu 7 nằm ở hàng 7 cột 7
- Hậu 8 nằm ở hàng 8 cột 4



Hình 3.5 – Thể hiện một phương án đặt 8 quân hậu trên bàn cờ kích thước 8x8.

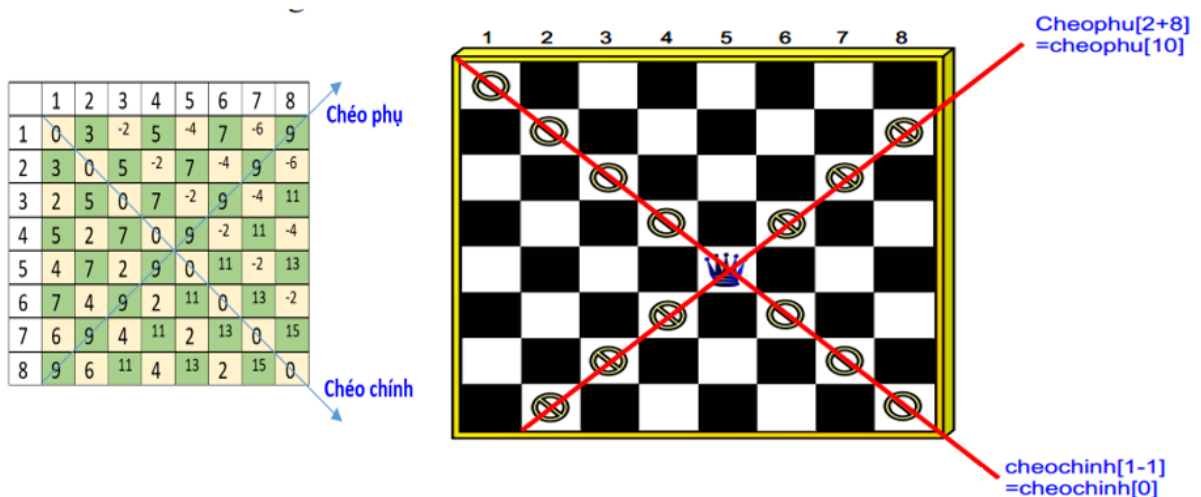
3.5.1 Cách 1: giải bằng phương pháp quay lui

Rõ ràng N quân hậu sẽ được đặt mỗi con một hàng vì hậu ăn được ngang, ta gọi quân:

- ✓ Hậu sẽ đặt ở **hàng 1** là quân **hậu 1**,
- ✓ Hậu ở **hàng 2** là quân **hậu 2**,
- ✓ ...
- ✓ Hậu ở **hàng n** là quân **hậu n**.

Vậy một nghiệm của bài toán sẽ được biết khi ta tìm ra được vị trí cột **j** của những quân hậu ở hàng thứ **i**.

- Một đường chéo chính bất kỳ sẽ đi qua một số ô, các ô đó có tính chất: **Hàng - Cột = C (Const)**. Với $1 - n \leq C \leq n - 1$.
- Một đường chéo phụ bất kỳ sẽ đi qua một số ô, các ô đó có tính chất: **Hàng + Cột = C (Const)**. Với $1 \leq C \leq 2n$.



Hình 3.6 - Thể hiện đường chéo chính và đường chéo phụ của bàn cờ kích thước 8x8.

❖ Ta có 3 mảng logic để đánh dấu:

- Mảng **Cot[1..n]**.
 - **Cot[j] = false** nếu như cột **j** còn tự do,
 - **Cot[j] = true** nếu như cột **j** đã bị một quân hậu không chế.
- Mảng **Cheophu[2n]**.
 - **Cheophu[i+j] = false** nếu đường **chéo phụ** ở các ô **(i,j)** còn tự do.
 - **Cheophu[i+j] = true** nếu đường **chéo phụ** ở các ô **(i,j)** bị không chế.
- Mảng **Cheochinh[2n]**.
 - **Cheochinh[i-j] = false** nếu như đường **chéo chính** ở các ô **(i,j)** còn tự do.
 - **Cheochinh[i-j] = true** nếu như đường **chéo chính** ở các ô **(i,j)** bị không chế.
- Ban đầu cả 3 mảng đánh dấu đều mang giá trị **false**. (Các cột và đường chéo đều tự do)
- ❖ **Thuật toán quay lui được thực hiện như sau:**

Xây dựng hàm đệ quy **Try(i)** với mục tiêu là thử gán đặt quân hậu thứ **i** (hàng **i**) bằng cột **j** tương ứng $x_i = j$ trong **vector** nghiệm **X** = [**x**₁, **x**₂, ..., **x**_n].

Bước 1: Kiểm tra điều kiện dừng

- Nếu **i == n+1**:
 - Tức là đã đặt đủ **n** quân hậu thành công.
 - **Ghi nhận một nghiệm hợp lệ** (tăng biến đếm số nghiệm hoặc in ra nghiệm).

Bước 2: Nếu chưa đạt điều kiện dừng

❖ Duyệt các lựa chọn đặt quân hậu

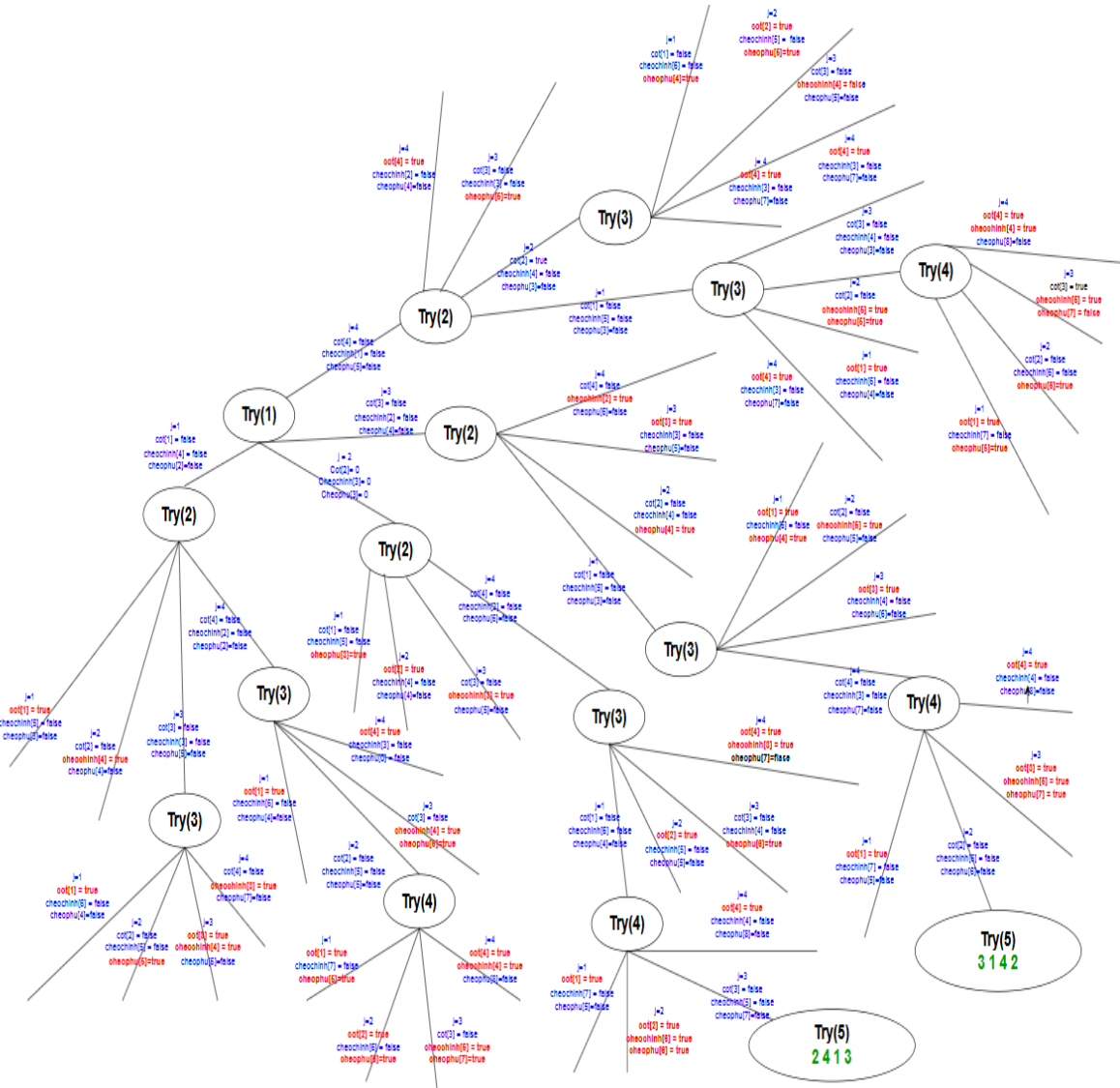
- Với mỗi cột **j** từ **1** đến **n**, ta xét khả năng đặt quân hậu tại ô **(i, j)**:
 - Kiểm tra nếu:
 - Cột **j** **chưa có quân hậu nào chiếm** (**Cot[j] == false**),
 - Đường chéo chính **(i - j)** **chưa bị chiếm** (**Cheochinh[i-j] == false**),
 - Đường chéo phụ **(i + j)** **chưa bị chiếm** (**Cheophu[i+j] == false**).
- Nếu thỏa mãn:
 - Gán **X[i] = j** (đặt quân hậu hàng **i** vào cột **j**).
 - Đánh dấu cột **j**, đường chéo chính **(i-j)**, và đường chéo phụ **(i+j)** là đã bị chiếm.
- **Gọi đệ quy: Try(i+1)** để tiếp tục đặt quân hậu tiếp theo.
 - **Quay lui (backtrack):** Sau khi hoàn tất lời gọi đệ quy **bỏ đánh dấu** cột **j**, đường chéo chính **(i-j)**, và đường chéo phụ **(i+j)**. Để có thể tiếp tục thử những lựa chọn khác cho quân hậu ở hàng **i**.

3.5.2 Chương trình tham khảo cách 1

```
#include <bits/stdc++.h>
using namespace std;
vector <int> X;
```

```
int N;
int dem=0;
vector<bool> Cot, Cheochinh, Cheophu;
void Try(int i){
    if (i==N+1) {
        dem++;
        /*cout<<dem<<". ";
        for (int z=1;z<=N;z++) cout<<X[z]<<" ";
        cout<<"\n";
        */
        return;
    }
    for (int j=1;j<=N;j++){
        if ((!Cot[j]) && (!Cheochinh[i-j+N]) && (!Cheophu[i+j])){
            X[i]=j;
            Cot[j]=true;
            Cheochinh[i-j+N]=true;
            Cheophu[i+j]=true;
            Try(i+1);
            Cot[j]=false;
            Cheochinh[i-j+N]=false;
            Cheophu[i+j]=false;
        }
    }
}
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("QUANHAU.INP", "r", stdin);
    freopen("QUANHAU.OUT", "w", stdout);
    cin>>N;
    X.resize(N+1);
    Cot.assign(N+1, false);
    Cheochinh.assign(2*N + 1, false);
    Cheophu.assign(2*N + 1, false);
    Try(1);
    cout<<dem;
    return 0;
}
```


3.5.3 Ví dụ minh họa cách 1



Hình 3.7 – Cây tìm kiếm quay lui trong bài xếp 4 hậu trong bàn cờ kích thước 4x4.

3.5.4 Độ phức tạp thời gian

- Số lần gọi đệ quy: $\frac{N!}{(K \cdot K!)}$
- Chi phí mỗi lần gọi: $O(K)$
- Tổng độ phức tạp: $O(\frac{N!}{K!})$
- Không gian bộ nhớ: $O(N)$

3.5.5 Cách 2: giải bằng phương pháp khử đệ quy dùng bitmask

Gọi:

- **state** đại diện cho trạng thái của bàn cờ tại một dòng: có tất cả $2^N - 1$ trạng thái.

- **row**: dòng hiện tại đang xét ($0 \rightarrow N-1$)
- **cols**: các cột đã bị quân hậu chiếm thể hiện bằng **bitmask**. mỗi bit từ **phải** sang trái tương ứng với 1 cột. **bit = 1** nghĩa là cột đó đã bị chiếm.
- **diag_cc**: các đường chéo chính bị chiếm. Có $(r - c)$ giống nhau $\rightarrow$ shift trái mỗi lần xuống dòng.
- **diag_cp**: các đường chéo phụ bị chiếm. Có $(r + c)$ giống nhau $\rightarrow$ shift phải mỗi lần xuống dòng.
- **X**: vector lưu cột mà quân hậu được đặt ở mỗi dòng.

Thuật toán hoạt động như sau:

Bước 1: Bắt đầu từ dòng 0, chưa có quân hậu nào.

Bước 2: Ở mỗi dòng, tính **cols_available** (các cột hợp lệ có thể đặt hậu) bằng:

$$\text{cols_available} = ((1 \ll N) - 1) \& \sim(\text{cols} | \text{diag_cc} | \text{diag_cp})$$

Ý nghĩa: $\text{cols_available} = ((1 \ll N) - 1) \& \sim(\text{cols} | \text{diag_cc} | \text{diag_cp})$;

✓ $((1 \ll N) - 1) = 11111111$

✓ **cols | diag_cc | diag_cp**: gộp lại tất cả các vị trí (cột và hai đường chéo) đã bị chiếm.

✓ $\sim(\text{cols} | \text{diag_cc} | \text{diag_cp})$: đảo bit để đánh dấu các vị trí còn trống.

✓ $((1 \ll N) - 1) \& \sim(\text{cols} | \text{diag_cc} | \text{diag_cp})$: tạo ra dãy bit có các bit 1 là các vị trí có thể đặt hậu.

Bước 3: Duyệt từng vị trí **bit = cols_available & -cols_available** (lấy bit phải nhất = 1 của **cols_available**).

Bước 4: Đặt hậu vào đó, cập nhật **bitmask** tương ứng với trạng thái mới (**row, cols, diag_cc, diag_cp, X**) đẩy trạng thái mới vào **stack**.

Bước 5: Xây dựng đến hàng cuối cùng (**row == N**) thì xuất kết quả

3.5.6 Chương trình tham khảo cách 2

```
#include <bits/stdc++.h>
using namespace std;

int N, dem = 0;

struct State {
    int row;
    int cols, diag_cc, diag_cp;
    vector<int> X;
};

string chuyen_NP(int n) {
    return bitset<8>(n).to_string();
}

int main() {
```

```

ios_base::sync_with_stdio(0);
cin.tie(0); cout.tie(0);
freopen("QUANHAU.INP", "r", stdin);
freopen("QUANHAU.OUT", "w", stdout);

cin >> N;
stack<State> ds_stack;

ds_stack.push({0, 0, 0, 0, vector<int>()});

while (!ds_stack.empty()) {
    State s = ds_stack.top();
    ds_stack.pop();

    int row = s.row;
    int cols = s.cols;
    int diag_cc = s.diag_cc;
    int diag_cp = s.diag_cp;
    vector<int> X = s.X;

    if (row == N) {
        dem++;
        cout << dem << ". ";
        for (int x : X) cout << x + 1 << " ";
        cout << "\n";
        continue;
    }
    int cols_available = ((1 << N) - 1) & ~(cols | diag_cc | diag_cp);
    while (cols_available) {
        int bit = cols_available & -cols_available;
        cols_available -= bit;
        int col = __builtin_ctz(bit);
        vector<int> X_next = X;
        X_next.push_back(col);

        ds_stack.push({
            row + 1,
            cols | bit,
            (diag_cc | bit) << 1,
            (diag_cp | bit) >> 1,
            X_next
        });
    }
}
return 0;
}

```

3.5.7 Ví dụ mô phỏng

Các giá trị ban đầu

- **N = 4**
- **cols = 0** (chưa cột nào bị chiếm)
- **diag_cc = 0** (chéo chính)
- **diag_cp = 0** (chéo phụ)
- **row = 0**
- **X = {}**

Bắt đầu đẩy trạng thái đầu tiên vào **stack**:

`ds_stack.push({0, 0, 0, 0, {}});`

Vòng lặp stack

Lần 1:

- row = 0, chưa có gì cả.
 - cols = 0000 = 0
 - diag_cc = 0000 = 0
 - diag_cp = 0000 = 0
- $\Rightarrow$ cols_available = (1111 & ~(cols | diag_cc | diag_cp)) = 1111
(4 bit: có thể đặt ở cột 0, 1, 2, 3)

Lần lượt xét từng cột từ **phải sang trái** vì **bit phải nhất** được chọn trước.

► Chọn cột 0 (bit = 0001) cols = 1

row = 1

cols = 0001 = 1

diag_cc = 0010 = 2

diag_cp = 0000 = 0

X = {0}

Cols	0	0	0	0
Bit	0	0	0	1
cols = cols bit = (0000 0001)	0	0	0	1
diag_cc = (diag bit)<<1 = (0000 0001)<<1	0	0	1	0
diag_cp = (diag bit)>>1 = (0000 0001)>>1	0	0	0	0

► Chọn cột 1 (bit = 0010) cols = 2

row = 1

cols = 0010 = 2

diag_cc = 0100 = 4

diag_cp = 0001 = 1

X = {1}

cols	0	0	0	0
bit	0	0	1	0
cols = cols bit = (0000 0010)	0	0	0	1
diag_cc = (diag bit)<<1 = (0000 0010)<<1	0	1	0	0
diag_cp = (diag bit)>>1 = (0000 0010)>>1	0	0	0	1

► Chọn cột 2 (bit = 0100) cols = 4

row = 1

cols = 0100 = 4

diag_cc = 1000 = 8

diag_cp = 0010 = 2

X = {2}

cols	0	0	0	0
bit	0	1	0	0
cols = cols bit = (0000 0100)	0	1	0	0
diag_cc = (diag bit)<<1 = (0000 0100)<<1	1	0	0	0
diag_cp = (diag bit)>>1 = (0000 0100)>>1	0	0	1	0

► Chọn cột 3 (bit = 1000) cols = 8

row = 1

cols = 1000 = 8

diag_cc = 0000 = 0

diag_cp = 0100 = 4

X = {3}

cols	0	0	0	0
bit	1	0	0	0
cols = cols bit = (0000 1000)	1	0	0	0
diag_cc = (diag bit)<<1 = (0000 1000)<<1	0	0	0	0
diag_cp = (diag bit)>>1 = (0000 1000)>>1	0	1	0	0

ds_stack hiện tại có 4

trạng thái tương ứng với 4 cách đặt hậu ở hàng 0.

ds_stack	0	1	2	3
	{1, 0, 2, 0, X={0}}	{1, 2, 4, 1, X={1}}	{1, 4, 8, 2, X={2}}	{1, 8, 0, 4, X={3}}

Lần tiếp theo — xử lý từng trạng thái từ ds_stack (theo kiểu LIFO – Last In, First Out vào sau ra trước)

Lấy phần tử ở đầu ds_stack: {1, 8, 0, 4, X={3}} Xét trạng thái với X = {3}

row = 1

$\Rightarrow$ cols_available = ~(1000 | 0000 | 0100) & 1111 = 0011 (có thể đặt ở cột 0, 1)

```
cols = 1000 = 8
diag_cc = 0000 = 0
diag_cp = 0100 = 4
```

► Đặt ở cột 0 (bit=0001) cols = 8

```
row = 2
cols = (1000|0001) = 1001 = 9
diag_cc = ((0000|0001)<<1) = 0001 << 1 = 0010 = 2
diag_cp = (0100|0001)>>1) = 0101 >> 1 = 0010
X = {3, 0}
```

Tiếp tục đến hàng 2, sau vài bước sẽ không còn cột hợp lệ → loại bỏ.

Lời giải hợp lệ đầu tiên

Cuối cùng, chương trình sẽ tìm được lời giải đầu tiên:

2. 3 1 4 2

=> (X = {2, 0, 3, 1}) // tương ứng quân hậu ở:

- Hàng 0, cột 2
- Hàng 1, cột 0
- Hàng 2, cột 3
- Hàng 3, cột 1

```
+---+---+---+---+
|   |   | Q |   |   |   ← Hàng 0
+---+---+---+---+
| Q|   |   |   |   |   ← Hàng 1
+---+---+---+---+
|   |   |   |   | Q|   ← Hàng 2
+---+---+---+---+
|   | Q|   |   |   |   ← Hàng 3
+---+---+---+---+
  0   1   2   3   ← Cột
```

Kết quả cuối cùng cho n = 4

Chỉ có 2 lời giải hợp lệ, in ra như sau:

```
1. 2 4 1 3
2. 3 1 4 2
```

3.5.8 Độ phức tạp thời gian

- Số lần gọi : $\frac{N!}{(K \cdot K!)}$
- Chi phí mỗi lần gọi: $O(K)$
- Tổng độ phức tạp: $O(\frac{N!}{K!})$
- Không gian bộ nhớ: $O(N)$

3.5.9 So sánh hai thuật toán

Thành phần	Thuật toán 1 (đệ quy + bool)	Thuật toán 2 (bitmask + stack)
------------	------------------------------	--------------------------------

Số lần gọi	$\frac{N!}{K \cdot K!}$	$\frac{N!}{K \cdot K!}$
Chi phí mỗi lần gọi	$O(K)$	$O(K)$
Tổng độ phức tạp	$O(\frac{N!}{K!})$	$O(\frac{N!}{K!})$
Không gian bộ nhớ	$O(N)$ (gọi đệ quy + 3 mảng bool cỡ $2 \cdot N$)	$O(N)$ (stack + bitmask gọn nhẹ)
⇒ Thuật toán đệ quy code ngắn gọn dễ hiểu hơn, thuật toán bitmask cần hiểu rõ các thao tác xử lý bit đối với số nguyên.		

Bảng 3.9 - So sánh hai thuật toán quay lui và bitmask trong bài toán quân hậu.

3.5.10 Test

<https://drive.google.com/file/d/1iYMCQo2euK7Jzt9QrinWAJpT5H0bXDZK/view?usp=sharing>

4. Một số bài toán khác trong sách giáo khoa chuyên tin quyển 1 [3]

Bài 1: Chọn k học sinh trong N học sinh - sgk chuyên tin quyển 1 trang 109.

Bài 2: Dãy tam phân $\{0, 1, 2\}$ độ dài N - sgk chuyên tin quyển 1 trang 109.

Bài 3: Các hoán vị khác nhau của xâu S - sgk chuyên tin quyển 1 trang 110.

Bài 4: Các xâu độ dài N chỉ gồm hai ký tự 'A' và 'B' mà không có 2 ký tự 'B' nào đứng cạnh nhau - sgk chuyên tin quyển 1 trang 110.

Bài 5: Chia dãy số thành K nhóm mà các nhóm có tổng bằng nhau - sgk chuyên tin quyển 1 trang 110.

Bài 6: Liệt kê tất cả các xâu con khác nhau của xâu S - sgk chuyên tin quyển 1 trang 110.

Bài 7: Liệt kê tất cả các cách khác nhau đặt N dấu ngoặc mở và N dấu ngoặc đóng đúng đắn - sgk chuyên tin quyển 1 trang 111.

Bài 8: Tìm số nguyên dương m nhỏ nhất - sgk chuyên tin quyển 1 trang 111.

Bài 9: Liệt kê tất cả các hoán vị khác nhau của xâu S - sgk chuyên tin quyển 1 trang 111.

Bài 10: Số siêu nguyên tố - sgk chuyên tin quyển 1 trang 112.

Bài 11: Một các chèn vào S các dấu '+' hoặc '-' - sgk chuyên tin quyển 1 trang 112.

Bài 12: Quân tượng - sgk chuyên tin quyển 1 trang 112.

...

5. Kết luận

Chuyên đề đã mang đến một góc nhìn toàn diện và thực tiễn về hai kỹ thuật nền tảng trong giải quyết bài toán tổ hợp: đệ quy quay lui (**Backtracking**) và **Bitmask**. Việc triển khai song song cả hai phương pháp trên cùng một bài toán không chỉ giúp học sinh làm quen với nhiều cách tiếp cận thuật toán khác nhau, mà còn rèn luyện

năng lực phân tích, đánh giá và lựa chọn chiến lược giải quyết phù hợp theo từng tình huống cụ thể.

Việc tiếp cận **Bitmask** – một công cụ lập trình có tính trừu tượng cao – tạo tiền đề vững chắc cho học sinh khi tiếp cận các chuyên đề nâng cao sau này như quy hoạch động theo trạng thái, thuật toán trên đồ thị hay thiết kế cấu trúc dữ liệu chuyên biệt. Cấu trúc trình bày mạch lạc, gồm phân tích tư duy, mã nguồn minh họa, đánh giá độ phức tạp và so sánh ưu – nhược điểm, góp phần hình thành một quy trình học tập bài bản và dễ tiếp thu.

Với 5 bài toán tiêu biểu, mỗi bài đều có hướng dẫn phân tích và chương trình minh họa chi tiết, chuyên đề không chỉ là tài liệu hỗ trợ học sinh trong quá trình học tập, mà còn khuyến khích khả năng tự học, tự phát triển lời giải và mở rộng sang các bài toán tương tự.

Đây là phiên bản đầu tiên của chuyên đề nên khó tránh khỏi những thiếu sót. Rất mong nhận được sự góp ý quý báu từ quý đồng nghiệp để nội dung chuyên đề ngày càng hoàn thiện, phục vụ tốt hơn cho việc dạy và học Tin học chuyên sâu trong nhà trường.

Tôi xin chân thành cảm ơn!

TÀI LIỆU THAM KHẢO

- [1] Vũ Quế Lâm, “Thao tác xử lý bit và ứng dụng (Bit Manipulation),” 16 11 2023. [Trực tuyến]. Available: https://viblo.asia/p/thao-tac-xu-ly-bit-va-ung-dung-bit-manipulation-3kY4gx19JAe#_i-tong-quan-ve-bit-va-xu-ly-bit-0.
- [2] Lê Minh Hoàng, Giải thuật và lập trình, NXB Đại Học Sư Phạm Hà Nội, 1999-2002.
- [3] Hồ Sĩ Đàm và các đồng sự, Tài liệu giáo khoa chuyên tin quyển 1, NXB Giáo dục Việt Nam, 2009.